life.augmented

AN5647

Application note

The Goertzel algorithm to compute individual terms of the discrete Fourier transform (DFT) in STM32 products

Introduction

This application note explains how to compute individual terms of the discrete Fourier transform using the Goertzel algorithm in STM32 products.

The Goertzel algorithm is derived and implemented as an iteration loop and as an IIR filter. A reference implementation based on the FFT is also included for verification.

The Goertzel algorithm is generalized to the non-integer frequency index case.

An application example is included: the dual-tone multi-frequency (DTMF) decoding. A MATLAB® script is used to generate a test signal and perform floating point decoding. A fixed-point C implementation is also provided.

1 General information

This document applies to the Arm®-based devices.

Note: Arm is a registered trademark of Arm Limited (or its subsidiaries) in the US and/or elsewhere.

1.1 Reference documents

- DT0085 *Coordinate rotation digital computer algorithm (CORDIC) test and performance verification*
- DT0087 *Coordinate rotation digital computer algorithm (CORDIC) to compute trigonometric and hyperbolic functions*
- DT0088 *FIR filter design by sampling, windowing and modulating the sinc() function*
- DB2956 *SensorTile development kit*
- DB2854 *Wearable sensor unit reference design for fast time to market*

2 The Goertzel algorithm

A set of N signal samples x() is transformed into a set of N frequency coefficients y() using the discrete Fourier transform (DFT): y(k) = sum of W(nk) x(n), where n and k go from 0 to N-1 and the twiddle factor W(t) is defined as exp (-j 2 π/N t).

The twiddle factor has the property that W(-Nk) = 1 for all k, therefore the k-th coefficient y(k) is the y(k) = sum of W (k (n-N)) x(n). The sum can be unfolded and manipulated to make it recursive:

y(k) = W(-k) x(N-1) + W(-2k) x(N-2) + W(-3k) x(N-3) + … + W(-k(N-1)) x(1) + W(-k N) x(0)

y(k) = W(-k) [x(N-1) + W(-k) x(N-2) + W(-2k) x(N-3) + … + W(-k(N-2)) x(1) + W(-k(N-1)) x(0)]

y(k) = W(-k) [x(N-1) + W(-k) [x(N-2) + W(-k) x(N-3) + … + W(-k(N-3)) x(1) + W(-k(N-2)) x(0)]]

y(k) = W(-k) [x(N-1) + W(-k) [x(N-2) + W(-k) [x(N-3) + … + W(-k(N-4)) x(1) + W(-k(N-3)) x(0)]]]

It is evident that the frequency coefficient y(k) can be computed in N steps using the following recursive formula: $y_n(k) = W(-k)\,[\,x(n) + y_{n-1}(k)\,]$, where n goes from 0 to N-1, $y_0(k) = 0$.

The recursive formula can be seen as a second order IIR filter because it is a weighted sum of the input sample x and the previous output y. Of course, only the final output $y_{N-1}(k)$ is of interest, intermediate outputs are not needed. It is possible to manipulate the coefficients of the filter so that the computation is done using real numbers, except for the last step where the final complex output $y_{N-1}(k)$ is computed.

The recursive formula in z-domain is:

$$Y(z) = W(-k)\left[X(z) + Y(z)z^{-1}\right]$$

Therefore the filter transfer function is:

$$H(z) = \frac{Y(z)}{X(x)} = \frac{W(-k)}{1 - W(-k)z^{-1}}$$

There is no change if the numerator and the denominator are multiplied by the same quantity $(1 - W(+k)\,z^{-1})$.

$$H(z) = \frac{W(-k)\left(1 - W(+k)z^{-1}\right)}{\left[\left(1 - W(-k)z^{-1}\right)\left(1 - W(+k)z^{-1}\right)\right]}$$

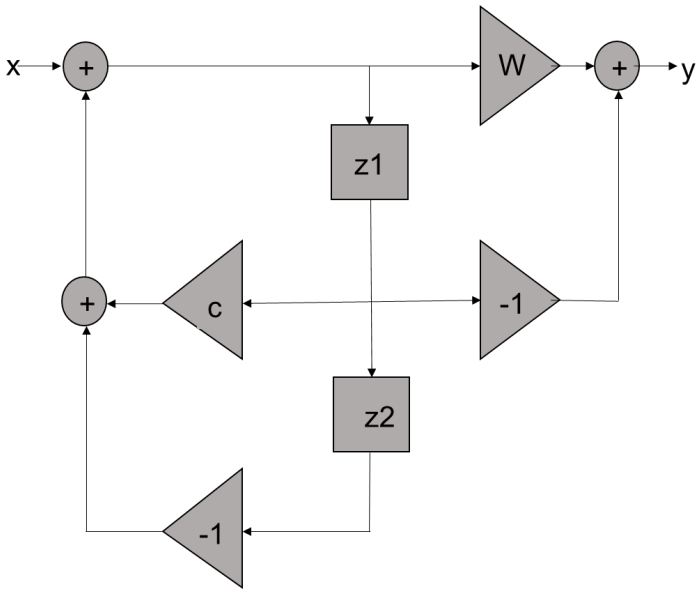
$$H(z) = \frac{\left[W(-k) - W(-k)W(+k)z^{-1}\right]}{\left[1 - \left(W(-k) + W(+k)\right)z^{-1} + W(-k)W(+k)z^{-2}\right]}$$

Remember that $W(-k) = \exp\frac{j2\pi}{Nk}$ and $W(+k) = \exp\left(\frac{-j2\pi}{Nk}\right)$, therefore $W(-k)W(+k) = \exp(0) = 1$ and $\left(W(-k) + W + k\right)) = 2\cos\left(\frac{2\pi}{Nk}\right)$

$$H(z) = \frac{W(-k) - z^{-1}}{1 - 2\cos\left(2\pi/Nk\right)z^{-1} + z^{-2}}$$

If the filter is implemented as direct II form (Figure 1), the status registers of the filter are real numbers, and multiplications and additions in the feedback loop are real. One complex multiplication is needed at the last step to compute the real and the imaginary part of the output y(k).

Figure 1. Direct II form for H(z) = (W – z – 1) / (1 – cz – 1 + z – 2), W = exp (j 2π/N k), c = 2cos(2π/N k)



2.1 Goertzel algorithm implementation

The following function takes a vector x made of N samples, and computes the k frequency coefficient (k from 0 to N-1). The real part is returned in I (in-phase) and the imaginary part is returned in Q (quadrature-phase).

```
function [I,Q] = goertzel(x,k,N)
  w = 2*pi*k/N;
  cw = cos(w); c = 2*cw;
  sw = sin(w);
  z1=0; z2=0; % init
  for n = 1 : N
    z0 = x(n) + c*z1 - z2;
    z2 = z1;
    z1 = z0;
  end;
  It = cw*z1 - z2;
  Qt = sw*z1;
  I = real(It) - imag(Qt); % I = It for real x because imag(Qt)=0
  Q = imag(It) + real(Qt); % Q = Qt for real x because imag(It)=0
```

2.2 Goertzel as an IIR filter

The following function returns the same results as the implementation shown above. Here the Goertzel algorithm is implemented as an IIR filter.

```
function [I,Q] = goertzelIIR(x,k,N)
  W = exp(1i*2*pi*k/N);
  c = 2*cos(2*pi*k/N);
  b = [W -1 0]; % FIR coefficients
  a = [1 -c 1]; % IIR coefficients
  y = filter(b, a, x);
  I = real(y(end));
  Q = imag(y(end));
```

2.3 Goertzel as the k-th coefficient of an N-point FFT

The following function returns the same results as the implementations shown above. Here the Goertzel algorithm is implemented as an FFT whose k-th coefficient is extracted.

```
function [I,Q] = goertzelFFT(x,k,N)
  y = fft(x);
  I = real(y(k+1));
  Q = imag(y(k+1));
```

2.4 Generalized Goertzel for k index not-integer

The Goertzel algorithm can be generalized to handle the case where the k index is not an integer. The computation is the same shown before. A correction step is added at the end to adjust the phase of the complex output.

Note: The magnitude of the output is changed.

```
function [I,Q] = goertzelgen(x,k,N)
  w = 2*pi*k/N;
  cw = cos(w); c = 2*cw;
  sw = sin(w);
  z1=0; z2=0; % init
  for n = 1 : N
    z0 = x(n) + c*z1 - z2;
    z2 = z1;
    z1 = z0;
  end;
  It = cw*z1 - z2;
  Qt = sw*z1;

  w2 = 2*pi*k;
  cw2 = cos(w2);
  sw2 = sin(w2);

  I =  It*cw2 + Qt *sw2;
  Q = -It*sw2 + Qt*cw2;
```

2.5 Generalized Goertzel equivalence to IQ demodulation

The generalized Goertzel algorithm is equivalent to IQ demodulation. For an N-point FFT, the frequency bin corresponding to coefficient k is centered at f = k * Fs/N, this is also the demodulation carrier to be used with IQ mixer. The time vector is n*1/Fs, n = 0 to N-1.

```
function [I,Q] = IQmixer(x,k,N)
  a = 2*pi*k/N*[0:N-1];
  c =  cos(a); % in-phase carrier
  s = -sin(a); % quadrature carrier
  I = sum(x.*c);
  Q = sum(x.*s);
end
```

2.6 Test for Goertzel algorithm implementations

The following script can be used to test the Goertzel algorithm implementations presented above. The computed numbers must be equal as indicated in the comments.

```
N = 64; % FFT length
x = randn(1,N) + 1i*randn(1,N); % random data
kf = rand(1)*(N-1); % k with fractional part
k = round(kf); % integer index to FFT coeff

[I1,Q1] =   goertzelC(x,k,N);
[I2,Q2] = goertzelIIR(x,k,N);
[I3,Q3] = goertzelFFT(x,k,N);
disp([I1 I2 I3]); % all equal
disp([Q1 Q2 Q3]); % all equal

[I4,Q4] = goertzelgen(x,kf,N);
[I5,Q5] =     IQmixer(x,kf,N);
disp([I4 I5]); % all equal
disp([Q4 Q5]); % all equal
```

2.7 Goertzel error bounds

The Goertzel algorithm is affected by an error bound proportional to N^2. The worst case is for k frequency index near 0 or near N-1. There is also a dependency on the input signal (e.g. a damped sinusoid may cause larger errors). As an example if the machine precision is $\varepsilon \sim 10^{-6}$, for N = 64 the error for the worst case can be as large as $\varepsilon \times 10^4$.

3 Application example: DTMF dual-tone multi-frequency signaling

The dual-tone multi-frequency signaling (DTMF) has been developed by the Bell System in the United States. It is standardized as ITU-T Rec Q.23. It is also known as “touch-tone” as tones are generated by pushing the buttons in the keypad of telephones.

Table 1. Dual-tone multi-frequency (DTMF)

Frequencies	1209 Hz	1336 Hz	1477 Hz	1633 Hz
697 Hz	1	2	3	A
770 Hz	4	5	6	B
852 Hz	7	8	9	C
941 Hz	*	0	#	D

Whenever a button is pressed, two tones are generated with the frequency indicated in the corresponding row and column. It would be inefficient to compute a full FFT to detect the presence of these tones. On the opposite, the Goertzel algorithm can be used to efficiently compute the frequency coefficients of the eight tones that can be present in the signal.

3.1 DTMF encoding and decoding, MATLAB script

In the following script, the sampling frequency is set to 4 kHz. N is set to 200, so that every 200 samples a new set of frequency coefficients is available. The minimum symbol duration is then N/Fs = 50 ms.

All possible symbols are encoded. The duration of each symbol is randomly selected in the range from N to 1.5 N. As the duration is variable, on the output some symbols may appear twice.

The signal is quantized to 8-bit and it is saved to file, so that the C implementation can run on the same input. Because of the quantization noise, on the output some symbols may be missing.

In the code the block of N samples is windowed, using a Hamming window, to enhance the signal-to-noise ratio of the coefficients. For every block, a set of eight frequency coefficients is computed. Coefficients that are above the threshold are selected. There should be one selected coefficient among the first four (row frequency), and another one among the last four (column frequency). If there are less or more than two selected coefficients, the symbol is not decoded and a blank space is printed instead.

```
% DTMF test, dual-tone multi-frequency
frow = [ 697,  770,  852,  941]; % frequencies for 1st tone
fcol = [1209, 1336, 1477, 1633]; % frequencies for 2nd tone
sym = ['1', '4', '7', '*', '2', '5', '8', '0', ... % symbols
       '3', '6', '9', '#', 'A', 'B', 'C', 'D' ];
symrow = [1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4]; % 1st tone for given symbol
symcol = [1 1 1 1 2 2 2 2 3 3 3 3 4 4 4 4]; % 2nd tone for given symbol
symmtx = [1 5 9 13; 2, 6, 10, 14; 3, 7, 11, 15; 4, 8, 12, 16]; % decoding matrix

Fs = 4000; % Hz, sampling frequency
N = 200; % minimum number of samples per symbol

x = []; % create test signal
symoutref = []; % reference for decoded output
for i=1:length(sym), % test each symbol
  Nsym = N + round(N/2*rand(1)); % samples for current symbol
  t = [0:Nsym-1]/Fs; % time vector
  x1 = sin(2*pi*frow(symrow(i))*t); % first tone
  x2 = sin(2*pi*fcol(symcol(i))*t); % second tone
  x = [x, x1+x2];
  symoutref = [symoutref, sym(i)];
end;
```

```
bits = 8;
Q = (max(x)-min(x))/(2^bits); % quantization step for 8 bit signal
x = round(x/Q); % some noise may also be added

fbin=Fs/N; % Goertzel frequency resolution, N must be high enough
k=round([frow fcol]/fbin); % N high enough so that k is different for each tone
if any(diff(k)==0), fprintf('same k index for different tones!\n'); return; end;
```

```
% Goertzel-based DTMF decoding
xblocks = floor(length(x)/N);
myspec = []; % zeros(xblocks,length(k));
for i = 1 : xblocks,
  i1 = (i-1)*N+1;
  i2 = i1+N-1;
  xt = x(i1:i2);
  xt = xt.*hamming(N)';
  for j = 1 : length(k),
    [I,Q] = goertzel(xt,k(j),N);
    myspec(i,j) = sqrt(I*I+Q*Q);
  end;
end;
```

```
th = max(myspec(:))/2; % threshold
myspecbin = myspec>th; % tone on/off detection
symout = [];
for i = 1 : xblocks,
  i1 = find(myspecbin(i,1:4)>0); if length(i1)~=1, i1=0; end; % 1st tone
  i2 = find(myspecbin(i,5:8)>0); if length(i2)~=1, i2=0; end; % 2nd tone
  if (i1==0) || (i2==0), symdec=' '; % no symbol decoded
  else                   symdec=sym(symmtx(i1,i2)); % symbol decoded
  end;
  symout = [symout, symdec]; % append decoded symbol
end;
```

```
% printout and plot
fprintf('reference string: %s\n',symoutref);
fprintf('decoded   string: %s\n',symout);

figure; imagesc(fbin*k/1000,[0:xblocks]*N/Fs*1000,myspec);
axis xy; axis([0 Fs/2/1000 0 length(x)/Fs*1000]); colorbar;
xlabel('Frequency (kHz)'); ylabel('Time (ms)');
title(sprintf('DTMF test, %d-bit Fs=%.1f kHz, %d-point Goertzel',bits,Fs/1000,N));

figure; imagesc(fbin*k/1000,[0:xblocks]*N/Fs*1000,myspecbin);
axis xy; axis([0 Fs/2/1000 0 length(x)/Fs*1000]); colorbar;
xlabel('Frequency (kHz)'); ylabel('Time (ms)');
title(sprintf('DTMF test, %d-bit Fs=%.1f kHz, %d-point Goertzel, th=%1.f',bits,Fs/
1000,N,th));

NFFT=128; NOVL=round(0.9*NFFT); WIN=hamming(NFFT);
figure; spectrogram(x,WIN,NOVL,NFFT,Fs);
title(sprintf('DTMF test, %d-bit Fs=%.1f kHz, %d-points FFT',bits,Fs/1000,NFFT));

% save quantized test signal to file to test C implementation
h=fopen('in.txt','wt'); fprintf(h,'%d\n',x); fclose(h);
```

The script plots the spectrogram of the generated signal (Figure 2), as well as the frequency coefficients computed by the Goertzel algorithm (Figure 3 and Figure 4). The coefficients above threshold drive the decoding.

Figure 2. Spectrogram of the generated test signal

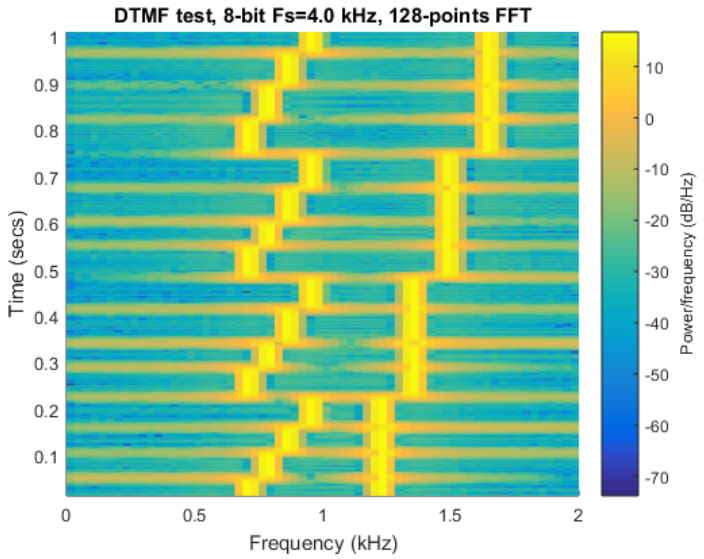


Figure 3. Frequency coefficients computed by the Goertzel algorithm

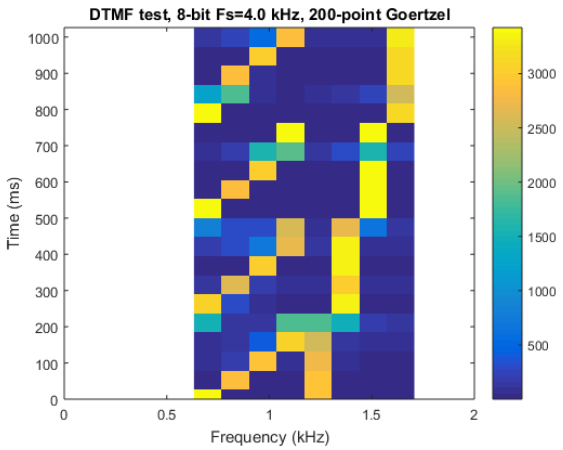
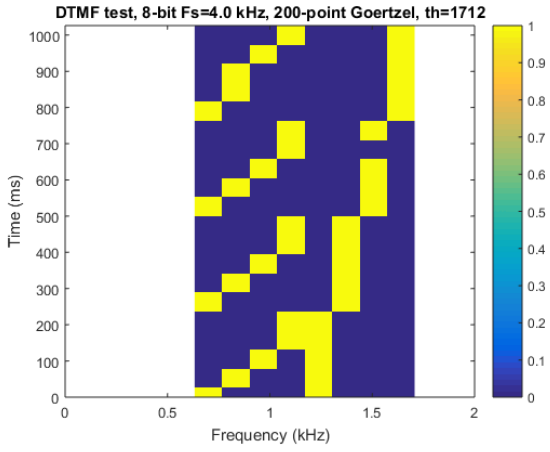


Figure 4. Frequency coefficients above threshold



The MATLAB output for a typical run is the following:

```
reference string: 147*2580369#ABCD
decoded   string: 147**25800369 #ABBCD
```

3.2 DTMF encoding and decoding, fixed-point C

The following C program performs the DTMF decoding using the Goertzel algorithm. The first argument on the command line is the name of the file with the signal samples. This file is generated by the MATLAB script shown in the previous section. The other arguments on the command line are the sampling frequency in Hz, the block length in samples, the number of bits for the quantization of the Goerzel constants, and the threshold to be used during the decoding process.

The computation of the Goertzel constants is based on sin() and cos(). These functions can be implemented in fixed-point using the CORDIC algorithm.

Note: For more informations about CORDIC algorithm refer to DT0087.

In the program the squared magnitude is computed, therefore a squared threshold must be used for decoding. The magnitude can also be computed and used: the sqrt() function must be implemented in fixed-point using the CORDIC algorithm.

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
```

```
#define MAXN 1000
```

```
// DTMF frequencies
int frow[4] = {  697,  770,  852,  941 }; // 1st tone
int fcol[4] = { 1209, 1336, 1477, 1633 }; // 2nd tone
```

```
// DTMF symbols
char sym[16] = { '1', '4', '7', '*', '2', '5', '8', '0',
                 '3', '6', '9', '#', 'A', 'B', 'C', 'D' };
```

```
// DTMF decoding matrix
int symmtx[4][4] = { { 0, 4,  8, 12 }, { 1, 5,  9, 13 },
                     { 2, 6, 10, 14 }, { 3, 7, 11, 15 } };
```

```
int win[MAXN]; // Window
```

```
// Goertzel
int c[8], cw[8], sw[8]; // Goertzel constants
int z1[8], z2[8]; // Goertzel status registers
int I[8], Q[8], M2[8]; // Goertzel output: real, imag, squared magnitude
```

```
int main(int argc, char *argv[]) {
  FILE *f;
  int i, Fs, N, b, x, n, z0, i1, i2, th;
  float w, S;
```

```
if(argc<6) { printf("usage: %s outfile Fs N bits threshold\n",argv[0]); return 0; }
  if(NULL==(f=fopen(argv[1],"rt"))) { printf("cannot read %s\n",argv[1]); return 0; }
  Fs=atoi(argv[2]); printf("Fs = %d Hz sampling frequency\n",Fs);
  N =atoi(argv[3]); printf("N  = %d points for Goertzel\n",N);
  if(N>MAXN) { printf("max N = %d\n",MAXN); fclose(f); return 0; }
  b =atoi(argv[4]); printf("b  = %d scaling is 2^b\n",b);
  S = (float)(1<<b); // scaling factor
  th=atoi(argv[5]); printf("th = %d threshold\n",th);
```

```
printf("\nreference string: ");
  for(i2=0;i2<4;i2++) for(i1=0;i1<4;i1++) printf("%c",sym[symmtx[i1][i2]]);
  printf("\ndecoded   string: ");
```

```
for(i=0;i<N;i++) { // init window (Hamming)
    win[i] = (int)round(S*(0.54 -0.46*cosf(2.0*M_PI*(float)i/(float)(N-1)))); }
```

```
  for(i=0;i<4;i++) { // init Goertzel constants
    // CORDIC may be used here to compute sin() and cos()
    w = 2.0*M_PI*round((float)N*(float)frow[i]/(float)Fs)/(float)N;
    cw[i] = (int)round(S*cosf(w)); c[i] = cw[i]<<1;
    sw[i] = (int)round(S*sinf(w));
    w = 2.0*M_PI*round((float)N*(float)fcol[i]/(float)Fs)/(float)N;
    cw[i+4] = (int)round(S*cosf(w)); c[i+4] = cw[i+4]<<1;
    sw[i+4] = (int)round(S*sinf(w)); }
```

```
for(n=0;!feof(f);) {
    i = fscanf(f,"%d",&x); if(i<1) continue;
    x = ((x*win[n])>>b);                                  // windowing
    if((n%N)==0) for(i=0;i<8;i++) { z1[i]=0; z2[i]=0; } // Goertzel reset
    // **** GOERTZEL ITERATION ****
    for(i=0;i<8;i++) {
      z0    = x + ((c[i]*z1[i])>>b) - z2[i];            // Goertzel iteration
      z2[i] = z1[i]; z1[i] = z0; }                      // Goertzel status update
    // **** GOERTZEL ITERATION ****
    n++; if((n%N)==0) { n=0; // finalize and decode
      for(i1=i2=-1,i=0;i<8;i++) {
        // CORDIC may be used here to compute atan2() and sqrt()
        I[i] = ((cw[i]*z1[i])>>b) - z2[i];              // Goertzel final I
        Q[i] = ((sw[i]*z1[i])>>b);                      // Goertzel final Q
        M2[i] = I[i]*I[i] + Q[i]*Q[i];                  // magnitude squared
        if(M2[i]>th) {                                  // DTMF decoding
          if(i<4) { if(i1==-1) i1=i;   else i1=4; }     // find 1st tone, one peak allowed
          else    { if(i2==-1) i2=i-4; else i2=4; }     // find 2nd tone, one peak allowed
        } }
      if((i1>-1)&&(i1<4)&&(i2>-1)&&(i2<4)) printf("%c",sym[symmtx[i1][i2]]);
      else printf(" ");
    } }
  printf("\n\n"); fclose(f); return 0;
}
```

The C output for a typical run on the same signal as the MATLAB is the following:

```
C:\>GoertzelDTMFdec in.txt 4000 200 12 2000000

Fs = 4000 Hz sampling frequency
N  = 200 points for Goertzel
b  = 12 scaling is 2^b
th = 2000000 threshold

reference string: 147*2580369#ABCD
decoded   string: 147* 25800369 #ABBCD
```

Revision history

Table 2. **Document revision history**

Date	Version	Changes
03-May-2021	1	Initial release.

Contents

1 General information 2
1.1 Reference documents 2
2 The Goertzel algorithm 3
2.1 Goertzel algorithm implementation 4
2.2 Goertzel as an IIR filter 4
2.3 Goertzel as the k-th coefficient of an N-point FFT 5
2.4 Generalized Goertzel for k index not-integer 5
2.5 Generalized Goertzel equivalence to IQ demodulation 5
2.6 Test for Goertzel algorithm implementations 6
2.7 Goertzel error bounds 6
3 Application example: DTMF dual-tone multi-frequency signaling 7
3.1 DTMF encoding and decoding, MATLAB script 7
3.2 DTMF encoding and decoding, fixed-point C 9
Revision history 13
Contents 14
List of tables 15
List of figures 16

List of tables

Table 1. Dual-tone multi-frequency (DTMF) 7
Table 2. Document revision history 13

List of figures

Figure 1. Direct II form for H(z) = (W – z – 1) / (1 – cz – 1 + z – 2), W = exp (j 2π/N k), c = 2cos(2π/N k) 4
Figure 2. Spectrogram of the generated test signal 9
Figure 3. Frequency coefficients computed by the Goertzel algorithm 9
Figure 4. Frequency coefficients above threshold 9

IMPORTANT NOTICE – PLEASE READ CAREFULLY

STMicroelectronics NV and its subsidiaries (“ST”) reserve the right to make changes, corrections, enhancements, modifications, and improvements to ST products and/or to this document at any time without notice. Purchasers should obtain the latest relevant information on ST products before placing orders. ST products are sold pursuant to ST’s terms and conditions of sale in place at the time of order acknowledgement.

Purchasers are solely responsible for the choice, selection, and use of ST products and ST assumes no liability for application assistance or the design of Purchasers’ products.

No license, express or implied, to any intellectual property right is granted by ST herein.

Resale of ST products with provisions different from the information set forth herein shall void any warranty granted by ST for such product.

ST and the ST logo are trademarks of ST. For additional information about ST trademarks, please refer to www.st.com/trademarks. All other product or service names are the property of their respective owners.

Information in this document supersedes and replaces information previously supplied in any prior versions of this document.

© 2021 STMicroelectronics – All rights reserved